AURA Research Freeze v1.0

Novelty boundary, formal verification contract, final architecture, and preregistered experimental blueprint

24 October 2026

Contents

AURA: Research Freeze and System Design Specification **1**

Executive decision . . . 1
1. Research question and falsifiable claim . . . 2
2. Scope and assumptions . . . 2
3. What prior work already covers — research differentiation matrix . . . 3
4. Formal verification semantics . . . 4
5. Final architecture and module contracts . . . 5
6. Research experiments — preregistered logic . . . 7
7. Baselines, validity and metrics . . . 8
8. Claim-to-evidence map . . . 8
9. Acceptance tests before writing the experiment engine . . . 9
10. Sequential engineering roadmap (pre-implementation) . . . 9
11. Research status and unresolved items . . . 10
12. Verified reference URLs and status . . . 10

AURA: Research Freeze and System Design Specification

Project: AURA — Agent Uncertainty and Residue Arbiter
Working study title: *Counterexample-Guided Evidence Certification for Multi-Step AI-Agent Task Completion*
Design status: Architecture baseline frozen for planning; *novelty, algorithmic advantage, and empirical superiority remain unproven*.
Version: v1.0, 24 October 2026
Purpose: Shareable pre-implementation contract for the research team, academic advisor, and future reproducibility audit.

Executive decision

AURA must independently decide whether **the entire contract-defined multi-step task was completed**, including all required outcomes and permitted side-effect boundaries. Successful tool invocations and a fluent agent completion message are not accepted as sufficient evidence. AURA will inspect state, recorded process evidence where required, and the limits of what can be observed. The initial implementation is a deterministic Python/SQLite/filesystem verification system; no LLM judge is required.

The proposed research differentiator is **not** state diffing, PASS/FAIL/UNCERTAIN labels, completion graphs, or selective verification alone. Those ideas overlap with substantial prior art. The candidate differentiator to test is a **counterexample-guided, obligation-aware inspection policy**,

coupled to *soundly scoped completion certificates*, evaluated at matched evidence availability, observation cost, and decision coverage. Even this combined idea has prior-art risk and must not be claimed novel before comparative experiments and a full-paper methods audit.

Operational requirement: AURA's decision concerns contract satisfaction, not whether the agent personally executed every tool. Tool traces matter only when the task requires a process condition or when they supply reliable evidence of an effect.

1. Research question and falsifiable claim

Primary RQ (RQ1): At matched contractual requirements, available evidence channels, and inspection budgets, does counterexample-guided inspection lower the complete cost of sound or appropriately abstaining task-completion verification relative to static, greedy, and targeted-evidence baselines?

RQ2: What complementary failures can state evidence, traces, and complete mutation accounting establish individually or only jointly?

RQ3: Which task-completion decisions are impossible to justify because successful and failing environment histories remain indistinguishable under the available evidence channels?

Main hypothesis (H1): A counterexample-guided inspection planner improves the *cost–error–coverage Pareto frontier* for heterogeneous multi-step task obligations, compared with strong static and greedy planners using the same authorized observations. **H1 is not assumed true.**

Secondary hypotheses: (H2) Capturing state and required process evidence exposes non-overlapping failure classes; (H3) explicit observation limits reduce unsupported PASS verdicts at the cost of abstention; (H4) contract mutation auditing exposes false-PASS bugs, including duplicate writes and authorized-footprint omissions.

Primary refutation criteria: No improvement over cost-greedy at matched decisive-coverage and false-PASS risk; any apparent gain disappears when snapshot costs are included; or held-out/externally sourced tasks do not reproduce the benefit.

2. Scope and assumptions

In scope (first scientific system): Explicit, human-reviewed tasks with machine-checkable obligations over a controlled relational database, files, and optional simulated external acknowledgements; immutable observations from independent read-only channels; fixed execution episodes; incomplete evidence; procedural rules explicitly named in contracts; evidence-cost accounting; reproducible injected and independently checked natural failures.

Out of scope: Understanding all implied user intent; proving correctness of arbitrary natural-language artifacts; preventing actions before they happen; automated rollback or recovery; actual money movement, live email sending, or production integrations; adversarial compromise of the independent verifier; proving that an observed prior read causally determined a downstream write without a trusted data-flow mechanism.

Trust boundary: We trust the *reviewed contract*, the declared environment model, verifier isolation, and authenticated observations captured within a declared scope. AURA's PASS is conditional on these assumptions. Under a model mismatch, missing mutation channel, stale snapshot, or tampered verifier, the guarantee does not hold.

Snapshot rule: No unqualified clean PASS for a no-collateral-effects obligation unless there is complete and consistent mutation accounting for the whole declared frame (e.g., a full initial-and-final snapshot under quiescence, or a separately validated complete mutation journal). Costs of capturing this evidence count toward total verification cost. Treat unknown external effects as unknown rather than implicitly clean.

3. What prior work already covers — research differentiation matrix

This table is a targeted pre-implementation audit based on source pages and abstracts. It is *not* a comprehensive reproduction or full-method audit of every paper.

Prior work	Verified mechanism / scope	Impact on AURA novelty	Our action
AppWorld (Trivedi et al., 2024)	Stateful agent benchmark with executable outcome evaluation and collateral-damage checks.	State/outcome verification and side-effect checks are established.	Reuse methodology; evaluate compatible frozen trajectories.
Agent-Diff (Pysklo et al., 2026)	Declarative state-diff contracts for enterprise API benchmarks (224 tasks reported).	Initial/final state diff and declarative outcome checks are not new.	Strong state-only baseline and interface reference.
AgentLTL (Elkoussy & Perez, 2026)	Trace constraints for procedure, ordering, and grounding.	Trace-based process verification is established.	Adapt only compatible process rules; do not claim original temporal logic.
EffectMatch (Zhang et al., 2026)	Controlled approval-to-persistent-effect validation, 206 tasks reported.	Persistent side-effect enforcement and effect boundaries have strong overlap.	Separate post-task certification from pre-commit operation gating.
Gao & Zhou (2026)	Evidence Pass/Fail/Unknown and evidence-supported bounds for benchmarks.	Three-valued evidential verdicts are established.	Cite; distinguish individual-run additional-evidence planning.
BAVAR / Strategic Verification (Adeli, 2026, non-peer-reviewed preprint)	Budgeted adaptive verifier selection mainly for long-running agent learning.	Adaptive allocation by cost/value is not original.	Compare *formulations*, not its illustrative numeric findings. **Its main tables explicitly say "not empirical findings."**
Can We Trust 'Done'? (Itkis, Sep 2026 preprint)	Completion judgment with bounded full/targeted additional-evidence requests, measured provider-read costs.	Nearest overlap for active evidence acquisition and abstention.	Obtain experiment artifacts; reproduce compatible targeted-request baseline or mark proxy accurately.
Interlock (Hao et al., Oct 2026 preprint)	Typed completion graphs with ground-ing/coupling/realization relations for task synthesis and training.	Obligation/evidence graphs and cross-resource requirements are not new per se.	Differentiate evaluation infrastructure from its task-synthesis/training goal.
Abrich verifier audit (2026 preprint)	Mutation audits find false accepts in AppWorld and WorkArena, notably duplicate effects in selected cases.	Contract hardening and mutation testing are already used.	Include count and frame mutation tests as a validity control, not a novelty claim.

Prior work	Verified mechanism / scope	Impact on AURA novelty	Our action
Trofimov & Novikov (2026)	Formal external-effect anomalies: retry, duplication, missing effects, compensation, etc.	External uncertainty and residue taxonomy already developed.	Use for fault taxonomy and honest observability limits.
Task-Relative Evidence Requirements for Agent Auditing (Sergeev, Oct 2026 IETF individual Internet-Draft)	Distinguishes evidence validity, task-relative sufficiency, and cross-domain evidence requirements; defines no implementation mechanism.	Evidence sufficiency and task-relative requirements as *concepts* are established.	Cite as *work in progress*, never a published IETF standard.
Classical CEGAR / active diagnosis	Counterexample-guided refinement, model-based fault isolation, information-seeking policies predate AI agents.	"Counterexample guidance" alone is not novel.	Restrict claim to measurable formulation and verified implementation for agent completion.

Highest novelty risk: BAVAR, Can We Trust 'Done'?, Interlock, and traditional model-based diagnosis/CEGAR methods. Their full methods and available artifacts require final reading before a publication novelty claim.

Publication language permitted today: "We *propose and evaluate* a counterexample-guided, contract-aware evidence selection method…"
Not permitted today: "first ever", "proven SOTA", "guarantees full human-intent satisfaction", "universal zero false accepts", or "optimal evidence selection" without precise assumptions and proof.

4. Formal verification semantics

4.1 World and contract

Let an execution world be `W = (S0, tau, ST, X)`: initial state `S0`, trusted recorded events `tau`, terminal state `ST`, and in-scope external effects `X`.

A task contract `C` consists of:

- `P`: required postconditions (values, multiplicity, cross-artifact consistency).
- `F`: permissible and impermissible changes (frame and invariant requirements).
- `Q`: explicitly mandatory process and provenance obligations (when relevant).
- `Omega`: declared observation surfaces, readback capabilities, trust, and limits.

`C(W)` is true **only if every required task obligation is satisfied**. No contract can prove unstated user intent. Not every task requires specific tool calls.

4.2 Evidence and compatible worlds

Let `M` be a model of allowed environment behavior, and `E` the observations collected by independent verifier probes. Define:

```
K(E) = { W in M : W is consistent with E }.
```

For a nonempty `K(E)`:

- **PASS** iff every `W` in `K(E)` satisfies `C`.
- **FAIL** iff every `W` in `K(E)` violates `C`.
- **UNCERTAIN** iff compatible successful and failing worlds both remain.

If `K(E)` is empty, the input evidence or environment model is inconsistent. The system **must not produce PASS or FAIL**. Emit top-level `UNCERTAIN` plus diagnostic `EVIDENCE_CONFLICT` / `MODEL_INCONSISTENT` and fail closed for downstream automation.

Bounded implementation equivalence:

- `PASS` if `(M AND E AND NOT C)` is UNSAT and `(M AND E)` is SAT.
- `FAIL` if `(M AND E AND C)` is UNSAT and `(M AND E)` is SAT.
- Otherwise, `UNCERTAIN` if both success and failure remain SAT.

An SMT solver returning `unknown`, a timeout, an incomplete abstraction, or unsupported predicate must lead to **UNCERTAIN**, not automatically PASS.

4.3 Guarantees and necessary limitations

Conditional soundness: If the actual world lies within the correctly modeled `K(E)`, and the solver, observations, and contract semantics are correct, PASS implies contract satisfaction. This is a property of the model and verified evidence, not user-intent equivalence.

Observational impossibility: If two worlds have identical results under every permitted observation but differ on `C`, no verifier restricted to those observations can always produce a correct decisive verdict on both. Our test suite must contain such pairs.

Global frame invariant: To certify "no unrelated changes," the evidence model must cover all mutable surfaces declared in the frame. Inspection of only the expected target rows does not suffice. Treat intermediate transient writes separately because `Diff(S0,ST)` alone cannot observe changes that were later reverted.

No causal-overclaiming: A trace showing a read and later write supports ordering and value consistency; it does not establish the agent genuinely used the read unless the environment enforces information-flow tracking.

5. Final architecture and module contracts

A. Specification

1. `ContractCompiler`: reviewed contract -> validated typed obligations, with stable IDs, cardinalities, allowed frames, temporal obligations, and required evidence clauses.
2. `BoundaryRegistry`: environment surface definitions -> authorized probes with scope, freshness, independence, failure behavior, and cost model.

B. Evidence

3. `EvidenceCapture`: controlled episode -> independent baseline/final snapshots, tool trace, optional separate mutation journal, and trusted timestamps/snapshot epoch.
4. `InspectorAdapters`: authorized read-only probe -> observation, content digest, source identity, consistency epoch, metering, or structured failure/denial.

C. Research core

5. `ObligationEvidenceModel`: contract + observations -> consistent-world constraints / conservative obligation abstractions, unresolved requirements, and testable candidate explanations.
6. `CERAPlanner`: model + probe catalogue + remaining budget -> next read-only probe or stop decision. Uses counterexample pairs to find discriminating probes but does not equate a single distinguished pair with global certification.

D. Arbitration

7. `VerdictArbiter`: proof/satisfiability checks -> PASS/FAIL/UNCERTAIN + reason codes; known contradictions never silently become PASS.
8. `CertificateWriter`: episode, model, evidence, probes and verdict -> immutable run certificate with contract hash and replay data.

Evaluation infrastructure (independent of AURA): `ExperimentHarness` owns world ground truth, hidden fault injection, test partitions, external dataset adapters, and statistical scoring. It never exposes truth or injected fault IDs to AURA.

5.1 Data contracts (interface freeze)

Object	Required fields	Invariant
`TaskContract`	id/version, obligation list, positive predicates, frame policy, process predicates, Omega	Immutable during evaluation; hash in certificate
`Obligation`	id, predicate, scope, severity, evidence requirement	Specific machine-checkable semantics, no free-text verdict rules
`ProbeSpec`	id, read-only target, scope, independence level, predicted coverage, cost definition	No user-state mutation; permissions known
`Observation`	probe_id, timestamp/epoch, source, result/failure, integrity digest, measured cost	Distinguish missing, negative, stale, and contradictory observations
`Certificate`	contract hash, run ID, evidence refs, obligations and verdicts, total cost, unresolved reasons	Every PASS supports every in-scope obligation under stated assumptions
`GroundTruthRecord`	world outcome, fault classes, independent audit details	Invisible to verifier until scoring

5.2 Evidence planner: design target, not asserted novelty

Finite-model reference mode: Explicitly enumerate small worlds and probe outcomes, compute the exact optimal adaptive decision tree where computationally feasible. Use this as a gold reference for planner quality; do not rely on it for large worlds.

Practical conservative mode: Maintain a set of unresolved obligations and a safe over-approximation of compatible worlds. Query for both satisfying and violating witnesses. If both exist, generate a candidate distinguishing probe whose result could eliminate relevant possibilities. Choose among such probes using conservative cost/coverage or a transparently trained development-only value model. Repeat until sound PASS, sound FAIL, an observational impossibility, or budget exhaustion.

Critical rule: A probe that distinguishes one counterexample pair may not resolve the entire verdict. Repeat reasoning after every observation; terminate only when the global contract decision has been established.

Probe-cost accounting: Capture time, read count, bytes transferred, CPU time, snapshot overhead, planning overhead, and timeout/denial costs. Compare equivalent end-to-end costs rather than pretending initial snapshot acquisition is free.

Practical fallback: If exact model construction is infeasible, a conservative deterministic postcondition/frame checker can provide a partial verdict, but must be explicit when evidence sufficiency is unproved. This is a separate operational mode, not silently equivalent to exact possible-world certification.

5.3 Technology freeze

- Python 3, `sqlite3`, filesystem hashes and canonical JSON for the local sandbox.
- Pydantic for validated task contracts; `pytest` and Hypothesis for tests.
- Z3 or a bounded exhaustive enumerator for exact possible-world reference tests.
- Simple Python tool-proxy traces first; OpenTelemetry adapter only when integration requires it.
- pandas/NumPy/SciPy/Matplotlib for analysis.
- No LLM in the PASS/FAIL decision path. Real LLM-produced execution trajectories are a later validation set, not the ground-truth oracle.

6. Research experiments — preregistered logic

E1. The planner mechanism (primary)

Create independently specified bounded problems whose complete worlds, outcomes and probe responses are known to the experiment harness. Compare:

1. Exhaustive inspection.
2. Static obligation-aware inspection.
3. Cost-greedy evidence cover.
4. Exact optimal adaptive decision policy (small-world reference).
5. CERA's counterexample-guided policy.
6. Available compatible targeted-evidence method from *Can We Trust 'Done'?* (if reproducible); otherwise a clearly labeled faithful or simplified proxy.

Fixed across conditions: Contract, probe catalogue, probe-response semantics, cost schedule, evidence permissions, initial evidence, and stop/abstain policy.

Main success criterion: Improvement on the *cost–false-PASS–decisive-coverage frontier*, not cheaper costs achieved by abstaining more. Include worst-case cost, not only mean.

E2. Workflow correctness and state/trace complementarity

Use four sandboxed task domains: academic records; e-commerce orders/refunds simulated locally; file/report management; access provision and audit. Compare state-only, trace-only and combined checking on frozen episode records. This **intentionally changes evidence type** and is a complementarity test, not a fair proof of superior planning logic.

Predeclare clean cases, equivalent valid paths, silence/no-op, partial writes, duplicate effects, wrong-target writes, unauthorized side effects, skipped mandatory validation, late cross-step regressions, transient reverted mutations, and source-data corruption.

E3. Epistemic boundary experiments

Construct paired worlds with different underlying outcomes but the same available observations. Vary external result readback, stale snapshot age, permission denial, source reliability, and evidence loss. Do **not** score UNCERTAIN as incorrect merely because the harness secretly knows the world outcome. Report separately:

- *Ontological correctness*: SUCCESS/FAILURE in hidden ground truth.
- *Epistemic correctness*: Which verdicts are warranted by the available evidence.

Measure false-PASS, false-FAIL, decisive coverage, selective risk and calibration only if a valid probabilistic model is used.

E4. External transfer

Inspect AppWorld and Agent-Diff task artifacts and licenses; choose compatible frozen agent trajectories. Run AURA over allowed task states without exposing hidden evaluation oracles to the verifier. Include honest accounting for adapter-specific instrumentation. External integration is a *milestone to establish*, not asserted feasible for every task today.

7. Baselines, validity and metrics

Baselines: Agent self-report (informational status-quo only), local per-tool readback, fixed state-only, fixed trace-only, fixed dual-surface, exhaustive, static set-cover, cost-greedy, exact bounded optimum, published targeted-evidence comparator when possible, and AURA CERA. Do not label a locally implemented proxy as the original author's code.

Primary metrics: False-PASS rate among actual failures; false-FAIL among actual successes; decisive coverage; selective error among decisive cases; full inspection cost including infrastructure; cost at matched false-PASS and coverage thresholds; exact-optimality gap on small instances. For external benchmarks also report per-task and per-domain performance.

Statistics: Paired episode comparisons; cluster by independently authored task template; uncertainty intervals (e.g., clustered bootstrap); predeclared primary comparison; correction for multiple comparisons; release raw counts and negative results. Sample size to be determined from development-only power analysis, not invented as proof of significance.

Threats: (i) privileged evidence access; (ii) circular fault generator and oracle; (iii) incomplete contracts and frames; (iv) hidden cost shifting into snapshots; (v) contaminated benchmark oracles; (vi) nonrepresentative cost assumptions; (vii) solver timeout or abstraction unsoundness.

Mitigation gates: Keep hidden truth out of verifier; freeze contracts before test faults; include benign variants and unseen fault compositions; use externally authored trajectories; report full cost; audit soundness assumptions; label uncertain and failed instrumentation explicitly.

8. Claim-to-evidence map

Intended paper claim	Required proof / experiment	Go/no-go evidence
Certifiability semantics are sound in the declared model	Formal proposition plus counterexamples for unsound variants	No unwarranted PASS in independently audited bounded test cases
CERA is cost-effective	E1 matched frontier vs greedy, exact-optimal and targeted-evidence comparator	Material advantage not explained by abstention or hidden preprocessing
Different evidence surfaces matter	E2 paired fault-class breakdown	Complementary detectable failures demonstrated with valid ground truth
UNCERTAIN is necessary under partial visibility	E3 indistinguishable pairs	Appropriate nondecisive judgments, with reason codes
Results generalize beyond synthetic faults	E4 external frozen trajectories	Consistent trends, with disclosed integration and selection biases
Certificates are reproducible	Evidence replay and contract/version checks	Independent replay matches decisions under same declared model

9. Acceptance tests before writing the experiment engine

1. **Complete outcome:** Correct row values, exact multiplicity and matching report -> PASS when frame and process evidence are sufficient.
2. **Missing report:** Correct database but absent required report -> FAIL, with specific violated obligation.
3. **Wrong row:** Target value right but unrelated row changed -> FAIL if declared frame fully observed.
4. **Duplicate insert:** Two matching records when one allowed -> FAIL.
5. **Unobservable webhook:** Local effects correct but essential external effect cannot be checked -> UNCERTAIN.
6. **Known bad plus unknown:** Wrong database update and ambiguous webhook -> FAIL (observed contract violation suffices).
7. **Conflicting evidence:** Mutually incompatible authenticated observations -> UNCERTAIN with evidence-integrity diagnostic, never PASS.
8. **Same final state, different required process:** Missing mandatory validation in reliable trace -> FAIL; if trace incomplete, UNCERTAIN.
9. **Reverted transient change:** Final diff clean but contract prohibits transient mutation -> only decisive with complete intermediate mutation record.
10. **Benign alternative path:** Different but contract-allowed sequence -> PASS when all other obligations are established.
11. **Unmonitored mutable surface:** Claimed no-side-effects frame spans unreadable store -> UNCERTAIN, not PASS.
12. **Solver unknown/timeout:** Safe abstention; never treat solver failure as proof.
13. **Cheaper evidence cannot prove frame:** Planner must acquire complete-scope evidence before PASS, even if costly.
14. **Budget exhausted:** UNCERTAIN with budget reason, and measured cost.
15. **Hidden-truth separation:** Altering hidden fault label alone cannot change AURA's decision under identical observations.

10. Sequential engineering roadmap (pre-implementation)

Gate 0 — This document: Freeze the project claim, scope, verdict semantics, interfaces, baseline fairness conditions, and acceptance tests. *Status: drafted; academic review recommended.*

Gate 1 — Formal specification: Build mathematical contract semantics and a finite-world reference test design; document snapshot consistency and mutation-scope completeness.

Gate 2 — Data and benchmark manifest: Specify domain fixtures, clean controls, held-out faults, probe catalogue, cost schedule and truth isolation. Freeze evaluation split before developing the main algorithm.

Gate 3 — Verification foundation: Implement contract compiler, snapshots, deterministic obligations, frame checking, trace rules, evidence recorder, and certificate export.

Gate 4 — Research planner: Implement exact-small reference, static/greedy baselines, and CERA; test semantic preservation after each observation.

Gate 5 — Evaluation: Execute paired E1–E3, perform mutational contract audits, compute costs and uncertainty intervals; adapt compatible external tasks for E4.

Gate 6 — Manuscript readiness: Report comparative novelty accurately, share experiment artifacts, and restrict claims to proven assumptions and measured outcomes.

No code, repository changes, or benchmark executions are authorized by this research-freeze document.

11. Research status and unresolved items

- **Architecture:** A1 - AURA contract/evidence/decision boundaries specified; implementation not started.
- **Novelty:** CONDITIONAL. Closely related targeted-evidence and adaptive verifier work exists; empirical and full-method differentiation is incomplete.
- **Verification theorem:** Mathematical proposition specified but not yet machine-checked or formally proved in a manuscript.
- **CERA algorithm:** Candidate; exact scoring and approximation analysis still open.
- **External benchmarks:** Identified but adapters, terms and available artifacts not yet audited.
- **Resource feasibility:** Local core should require no paid API/GPU, but external benchmark scale and exact model-solving overhead need measurement.

12. Verified reference URLs and status

1. Trivedi et al., **AppWorld** (ACL 2024): https://arxiv.org/abs/2407.18901 ; code: https://github.com/StonyBroo
2. Pysklo et al., **Agent-Diff** (Feb 2026 preprint): https://arxiv.org/abs/2602.11224 ; code: https://github.com/agent-diff-bench/agent-diff
3. Elkoussy & Perez, **AgentLTL** (Jul 2026 preprint): https://arxiv.org/abs/2607.02599
4. Zhang et al., **EffectMatch** (Sep 2026 preprint, submitted to TSE): https://arxiv.org/abs/2609.31301
5. Gao & Zhou, **Evidence-Supported Bounds** (May 2026 preprint): https://arxiv.org/abs/2605.10448
6. Adeli, **Strategic Verification / BAVAR** (Aug 2026 *non-peer-reviewed* preprint; numeric result tables explicitly illustrative): https://www.preprints.org/manuscript/202608.2057
7. Itkis, **Can We Trust 'Done'?** (Sep 2026 preprint; authored by associated product organization): https://agentaction.dev/research/completion-assessment
8. Hao et al., **Interlock** (Oct 2026 preprint): https://arxiv.org/abs/2610.19686
9. Abrich, **Finding Blind Spots in AppWorld and WorkArena** (Oct 2026 preprint): https://arxiv.org/abs/2610.09142
10. Trofimov & Novikov, **When Tool Calls Succeed but Workflows Fail** (Sep 2026 preprint): https://arxiv.org/abs/2609.15397
11. Bellibatlu et al., **Executable-Contract Audit** (Sep 2026 preprint): https://arxiv.org/abs/2609.37315
12. Sergeev, **Task-Relative Evidence Requirements for Agent Auditing** (*individual IETF Internet-Draft, work in progress, not a standard*, Oct 2026): https://www.ietf.org/ietf-ftp/internet-drafts/draft-sergeev-audit-task-requirements-00.html
13. Classical **counterexample-guided abstraction refinement** (CEGAR): https://doi.org/10.1007/10722167
14. Open-source infrastructure: https://github.com/pydantic/pydantic ; https://github.com/HypothesisWorks/hy ; https://github.com/Z3Prover/z3 ; https://github.com/open-telemetry/opentelemetry-python

Source provenance: This specification synthesizes the user-provided 33-page *Independent Research Evaluation and Architectural Critique: AURA* and the previous research discussions, supplemented by independently consulted online abstracts and project pages on 24 October 2026. Some cited works have not undergone a line-by-line full-text audit. Their presence in the bibliography does not establish that their findings have been independently reproduced. Claims marked as hypotheses are proposed by this research team, not results of completed experiments.